

Timer Lawn: Solving the Timer Wheel Overflow Problem for Large Scale and High Throughput Systems

Adam Lev-Libfeld

NextSilicon Ltd.

Tel-Aviv, Israel

adam.lev.libfeld@nextsilicon.com

Abstract—Timer facilities sit at the core of operating systems[1], network protocol implementations, real-time and stream processing, and several database capabilities. This paper presents Lawn, a field-tested timer data structure that maps queues by time-to-live (TTL) rather than by expiration time, so that its span is unbounded and it never rebuilds on overflow. We give the algorithm and its analysis, and evaluate an allocation-free implementation (Lawn2) against a production tickless hierarchical timing wheel and a naive single-level wheel. When the number of distinct TTLs is small relative to the number of live timers, Lawn2 deletes about 3.8 times faster at scale, inserts about 1.4 to 1.6 times faster at scale, uses about 1.8 times less memory per timer, and holds memory flat across an unbounded TTL span. It also drains due timers faster than the wheel under every realistic TTL distribution we tested, by 3.8 to 20.7 times depending on how TTLs cluster, and keeps the idle per-tick check at $O(1)$ (about 1 ns versus the wheel’s cascade-bound $\sim 6.6 \mu\text{s}$ at one million timers). These advantages are contingent: once the distinct-TTL count grows past roughly 2–6% of the live-timer count, per-tick cost becomes $O(t)$ and Lawn loses to the wheel. Even on the winning side of this boundary, “small” is relative rather than tiny in absolute terms: at one hundred thousand live timers the crossover sits near two thousand two hundred distinct TTLs, a range a configuration-driven or otherwise bounded TTL set can plausibly stay under, though we did not measure the actual distinct-TTL fraction of any real deployment directly. We state this boundary quantitatively and report where Lawn should and should not be used.

Keywords—Stream Processing, Timing Wheel, Dehydrator, Callout facilities, protocol implementations, Timers, Timer Facilities, Lawn.

I. Introduction

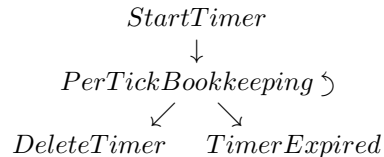
This paper presents a timer data structure designed for high-throughput computer systems, called Lawn. The current state-of-the-art algorithm, the Timing Wheel, is asymptotically optimal, but under some use cases (namely where the maximum TTL is unpredictable, or the needed tick resolution may change) it under-performs due to the overflow problem, which Lawn addresses. Lawn improves flexibility in TTL and tick resolution with no need for prior knowledge of the using system, under the single requirement that timer TTLs are drawn from a bounded set of discrete values. We make both a theoretical and an

empirical case, and we are explicit about the regime in which the empirical advantage holds.

The TTL-keyed queue-map idea itself, and the name Lawn, were introduced in an earlier preprint by the same author [2]. What is new here is the evaluation: an allocation-free implementation (Lawn2), a differential correctness gate across four independently written implementations, and a directly measured t/N crossover map (Section VII) showing exactly where the earlier design’s advantage holds and where it does not. Section VII also reports several results the earlier version did not: memory behavior at scale, workload characterization, and a robustness sweep to hundreds of millions of timers.

A. Model

In a similar manner to previous work[3], [4], [5], the model discussed in this paper consists of the following components, each corresponding with a different stage in the life cycle of a timer in the data store:



a) *StartTimer*(TTL,timerId,Payload): This routine is called by the client to start a timer that will expire after the TTL has passed. The client is also expected to supply a timer ID in order to distinguish it from other timers in the data store. Some implementations also allow the client to provide a Payload, usually some form of a callback action to be performed or data to be returned on timer expiration.

b) *PerTickBookkeeping*(*t*): This routine encompasses all the actions, operations and callbacks to be performed as part of timer management and expiration check every interval as determined by the data store granularity. Upon discovery of an outstanding timer to expire *TimerExpired* is initiated by this routine.

c) *DeleteTimer*(timerId): The client may call this utility routine in order to remove from the data store an

outstanding timer (corresponding with a given timer ID), before PerTickBookkeeping has marked it to be expired.

d) TimerExpired(timerId): Internally invoked by either PerTickBookkeeping or DeleteTimer, this routine entails all actions and operations needed to remove all traces of the timer corresponding with a given timer ID from the data store, and to invoke any callbacks that were provided as Payload during the StartTimer routine.

B. Callback run-time complexity

Since payload and callback run-time complexity varies between implementations, and the store of such data can be achieved in amortized $O(1)$ using a hash map with callback handling done in a discrete, highly (or even embarrassingly) parallel manner, this paper disregards this aspect of timer stores.

II. Related Work

Timer implementations have been extensively studied, with a focus on efficiency, scalability, and correctness. This section reviews the key developments relevant to this work.

A. Timer Data Structures

The most influential work on timer data structures is the Timing Wheel algorithm proposed by Varghese and Lauck [6]. In the common case it provides $O(1)$ operations for timer insertion, deletion, and expiration, which is asymptotically optimal. However, as discussed below, it suffers from the overflow problem in scenarios with unpredictable TTL ranges.

Several extensions address the overflow problem. The Hierarchical Timing Wheel [3] uses multiple levels of wheels to handle a wider range of TTLs, at the cost of additional complexity, periodic cascade work, and memory overhead. The Multi-Pass approach handles overflow by tracking the number of wheel cycles for each timer, but can lead to $O(n)$ complexity for PerTickBookkeeping in the worst case. A separate line of work, Calendar Queues [4], addresses a related priority-queue problem by hashing on absolute expiration time into a resizable bucket array rather than TTL duration, sharing the model’s general shape but not its overflow-avoidance mechanism. The general principle of bucketing by a discretized TTL to bound storage has since appeared independently outside the timer-facility literature: Time-Bucketed Balance Records [7] applies the same coalesce-within-a-bucket idea to bound per-account storage for TTL-bearing tokens in resource-constrained blockchain systems, a different domain and a different storage model, but the same underlying mechanism.

B. Concurrent Timer Implementations

Concurrent timer implementations are an active research area, driven by the need for scalable timer facilities in multi-threaded and distributed systems. [8] presents a scalable timer implementation for large-scale systems, while [9] proposes an auto-scaling timer implementation

for cloud environments. Lock-free and RCU-based timer implementations have been proposed to minimize contention, and NUMA-aware concurrent priority queues [10] and NUMA-aware scheduling more broadly [11] address the same class of contention and locality problem for related structures. We note that Lawn, as evaluated here, does not itself implement a lock-free timer, and NUMA effects are not evaluated (Section VII reports single-machine, non-NUMA concurrency results only). Concurrency is discussed as an architectural property and left as future work (Section IV-E).

C. Distributed Timer Implementations

The nature of the algorithms studied here lends itself to distribution to edge nodes in local applications, requiring no centralized timer management. Hence, distributed implementations are derived naively and are not discussed further.

III. Current Solutions

A. List and Tree Based Schemes

Operation	List Based	Tree Based
StartTimer	$O(n)$	$O(\log(n))$
PerTick	$O(1)$	$O(1)$
DeleteTimer	$O(n)$	$O(\log(n))$

TABLE I

Runtime Complexity for common data structure based schemes

Being included as an integral part of almost any modern programming language, these basic data structures enable convenient addition of timer management to any software. That said, such simple structures are appropriate only for narrow use cases, where the number of timers is small or the ticks are far enough apart. The complexities above are the textbook figures for a sorted list and a balanced tree. They are not to be confused with the unsorted-append naive baseline benchmarked in Section VII, which is a different data structure.

B. Hashed Timing Wheel

Operation	Worst	Mean
StartTimer	$O(n)$	$O(1)$
PerTick	$O(n)$	$O(1)$
DeleteTimer	$O(1)$	$O(1)$

TABLE II

Runtime Complexity for the Hashed Timing Wheel scheme

The Hashed Timing Wheel was designed as an all-purpose timer storage solution for a unified system of known size and resolution[6], [3], [12], [8], [9]. In ideal conditions its run-time complexity is optimal, but real-world implementations are bound by the maximal TTL and resolution combination, or require a costly $O(n)$ run-time rebuild upon reaching such limits, commonly referred to as “the overflow problem.” For large numbers of timers, the common workaround is to over-estimate

the needed resolution and/or TTL so as to postpone rebuilding (resizing) for as long as possible.

C. Overflow Algorithms

To handle overflows correctly there are two valid options.

1) Increase current wheel slots (Alg. 1): Resize the overflowing wheel so it includes a larger number of slots into the future. This entails a worst-case cost of $O(n)$ upon any addition of an overflowing timer.

Algorithm 1 Timer Wheel Resize

```

1: function StartTimer( $id, ttl, payload$ )
2:   if  $ttl > len(Wheel)$  then
3:     MissingSlots  $\leftarrow ttl - len(Wheel)$ 
4:     ExtWheel  $\leftarrow$  array of length  $len(Wheel) +$ 
       MissingSlots
5:     for Slot, SlotIndex in Wheel do
6:       NewSlotIndex  $\leftarrow (len(Wheel) + SlotIndex)$ 
       mod  $len(ExtWheel)$ 
7:       ExtWheel[NewSlotIndex]  $\leftarrow$  Slot
8:     Wheel  $\leftarrow$  ExtWheel
9:     slot  $\leftarrow (current\ time + ttl) \bmod len(Wheel)$ 
10:    append  $payload$  to Wheel[slot]
11:    TimerHash[ $id$ ]  $\leftarrow (payload, slot)$ 
```

2) In-place addition of cycle count for each item (Alg. 2): Each element in a slot is stored with the number of wheel cycles needed before expiration. This entails a worst-case cost of $O(n)$ for the PerTickBookkeeping operation.

Algorithm 2 Timer Wheel Multi-Pass (in-place)

```

1: function StartTimer( $id, ttl, payload$ )
2:    $T \leftarrow (payload=payload, cycles=(current\ time+ttl)$ 
    $\div len(Wheel))$ 
3:   slot  $\leftarrow (current\ time+ttl) \bmod len(Wheel)$ 
4:   append  $T$  to Wheel[slot]
5:   TimerHash[ $id$ ]  $\leftarrow T$ 

6: function PerTickBookkeeping
7:   current slot  $\leftarrow$  current slot + 1
8:   for element in current slot do
9:     element.cycles  $\leftarrow$  element.cycles-1
10:    if element.cycles  $\leq 0$  then
11:      TimerExpired( $T_{id}$ )
```

IV. The Lawn Data Structure

A. Intended Use Cases

This algorithm was first developed during the construction of a large-scale, Stream Processing geographic intersection product using a FastData[13] model. The data structure receives inputs from one or more systems that make use of a very limited range of TTLs in proportion to the number of concurrent timers they use.

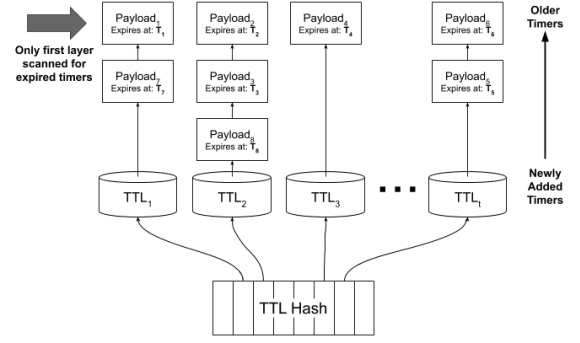


Fig. 1. A schematic view of the data structure components.

a) Assumptions and Constraints: Lawn is intended for use under the assumption that

Unique TTL Count $\ll$ Concurrent Timer Count.

This is the load-bearing assumption of the whole approach. When it holds, TTL bucketing yields the results below. When it fails, Lawn degrades to $O(n)$ per tick and loses to the wheel. This is a design-level argument rather than a measured claim: when TTLs originate from an algorithmic or configuration-driven source, such as a fixed backoff schedule or a small number of cache-tier classes, the set of possible values is bounded by construction and does not grow with the number of live timers, so the assumption is plausible for such systems. We did not instrument any of the deployments discussed in Section VIII-A to confirm their actual distinct-TTL fraction, and flag this as an open verification rather than an established result. Section VII quantifies exactly where the crossover lies.

1) The Data Structure: Lawn is, at its core, a hash of sorted sets¹, much like the Timing Wheel. The main difference is the key used for hashing these sets: the timer TTL. Different timers are stored in the same set based only on their TTL, regardless of arrival time. Within each set, the timers are naturally sorted by time of arrival, effectively using the set as a queue (fig. 1). Because two timers in the same bucket share a TTL, any timer already in the queue will expire before a newly added one, so the queue stays sorted by expiration for free.

The data structure is analogous to blades of grass (hence the name): each blade grows from the roots up, and periodically (every *Tick*) the overgrown tops (the expired timers) are maintained by mowing the lawn to the desired level (current time).

B. Algorithm

Note the *closest expiration* guard in PerTickBookkeeping: a tick on which nothing is due returns in $O(1)$ without scanning the buckets. This empty-tick fast path is important in practice, because most ticks expire nothing.

¹These are the TTL ‘buckets’

Algorithm 3 The Lawn Data Store

Precondition:

```

1: id - a unique identifier of a timer.
2: tll - a whole product of TickResolution representing
   the amount of time to wait before triggering the given
   timer payload action.
3: payload - the action to perform upon timer expiration.
4: current time - the local time of the system as a whole
   product of TickResolution

5: function InitLawn()
6:   TTLHash  $\leftarrow$  new empty hash set
7:   TimerHash  $\leftarrow$  new empty hash set
8:   closest expiration  $\leftarrow \infty$ 

9: function StartTimer(id, tll, payload)
10:  endtime  $\leftarrow$  current time + tll
11:  T  $\leftarrow$  (endtime, tll, id, payload)
12:  TimerHash[id]  $\leftarrow T$ 
13:  if tll  $\notin$  TTLHash then
14:    TTLHash[tll]  $\leftarrow$  new empty queue
15:  TTLHash[tll].insert(T)
16:  if endtime < closest expiration then
17:    closest expiration  $\leftarrow$  endtime

18: function PerTickBookkeeping()
19:  if current time < closest expiration then
20:    return
21:  closest expiration  $\leftarrow \infty$ 
22:  for queue  $\in$  TTLHash do
23:    T  $\leftarrow$  peek(queue)
24:    while T.endtime < current time do
25:      TimerExpired(T.id)
26:      T  $\leftarrow$  peek(queue)
27:    if T.endtime < closest expiration then
28:      closest expiration  $\leftarrow T$ .endtime

29: function TimerExpired(id)
30:  T  $\leftarrow$  TimerHash[id]
31:  DeleteTimer(T)
32:  do T.payload

33: function DeleteTimer(id)
34:  T  $\leftarrow$  TimerHash[id]
35:  if T.endtime = closest expiration then
36:    closest expiration  $\leftarrow \infty$ 
37:  TTLHash[T.tll].remove(T)
38:  TimerHash.remove(T)
39:  if TTLHash[T.tll] is empty then
40:    TTLHash.remove[tll]

```

C. Two Implementations: Reference Lawn and Lawn2

The same algorithm admits two C implementations. They produce identical expiry schedules (verified by a differential test in Section VII) and differ only in who owns the timer node and how a timer is addressed for cancellation.

Reference Lawn favors readability and key addressing. The store owns each timer node, copies the client's key, and indexes the timers in a hash map, so a timer can be cancelled by an arbitrary opaque key. That convenience costs one node allocation, one key copy, and two hash-map operations per insertion, which makes the reference implementation slower and heavier than a production timing wheel (Section VII). We present it as the direct expression of the algorithm.

Lawn2 keeps the algorithm but removes the per-operation allocation. Its nodes are intrusive and caller-owned (embedded in the client object), so an insertion neither allocates memory nor copies a key. Deletion is $O(1)$ through the node's own links, which lets Lawn2 drop the element-id hash map, and the TTL $\rightarrow$ queue map uses open addressing with no per-entry allocation. The performance claims in this paper refer to Lawn2.

1) Correctness & Completeness: To prove correctness, we show that for each timer t with TTL tll , TimerExpired is called on t within one Tick of tll . Since each timer is stored exactly once in its corresponding bucket and buckets are independent, it suffices to demonstrate correctness within a bucket:

$$\forall T^{start}, T^{tll} \in \mathbb{N} \quad \exists T^{stop} \in \mathbb{N} : T^{stop} - T^{start} \approx T^{tll}$$

Alternatively, using the sorting analogy of G. Varghese et al.[3], given two triggers T_n, T_m :

$$\forall T_n, T_m \mid T_n^{start} < T_m^{start}, T_n^{tll} = T_m^{tll} \quad \exists T_n^{stop}, T_m^{stop} \Rightarrow T_n^{stop} < T_m^{stop}$$

Since each bucket contains only triggers with the same TTL, this simplifies to

$$\forall T_n, T_m \mid T_n^{start} < T_m^{start} \Rightarrow T_n^{stop} < T_m^{stop}$$

which, the bucket being a sorted set ordered by T^{start} and triggers being expired from old to new, is self-evident.

2) Formal Correctness Proof: Let T be a timer with TTL tll started at time t_{start} . We prove T is expired at $t_{start} + tll$. Let Q_{tll} be the queue of timers with TTL tll , $t_{current}$ the current time, and $t_{expiry} = t_{start} + tll$.

Base Case: At t_{start} , T is inserted into Q_{tll} with $t_{expiry} = t_{start} + tll$.

Inductive Step: Assume at $t_{current} < t_{expiry}$ that T is in Q_{tll} . If $t_{current} + 1 < t_{expiry}$, then T remains, as PerTickBookkeeping only expires timers with $t_{expiry} \leq t_{current} + 1$. If $t_{current} + 1 \geq t_{expiry}$, then T is expired. By induction, T is expired at t_{expiry} .

3) Complexity: The Lawn data structure is dense by design: every timer is stored exactly once, a new trigger adds at most one TTL bucket, and empty buckets are

removed. The footprint therefore grows linearly with the number of timers, so space is $O(n)$ (confirmed empirically in Section VII).

Operation	Worst	Mean
StartTimer	$O(1)$	$O(1)$
PerTick	$O(n)$	$O(t \sim 1)$
DeleteTimer	$O(1)$	$O(1)$

TABLE III
Runtime Complexity for the Lawn scheme

StartTimer and DeleteTimer are $O(1)$ amortized. The worst case is $O(n)$ on a hash-map/table resize, as is standard for hash-based structures (visible in the tail of the measured insert latency). An empty tick is $O(1)$ via the *closest expiration* guard. A tick that does expire timers scans the head of each of the t live TTL buckets, so its cost is $O(t + k)$ for k expired timers, with a worst case of $O(n)$ (all timers due in one tick). When t is bounded (i.e. $t = O(1)$, the assumption of Section 4.A), the per-tick cost is $O(1)$. Otherwise it is $O(t)$ and Lawn is worse than a wheel. This dependence on t is the algorithm’s defining trade-off and is measured directly in Section VII.

a) Space: is $O(n)$: at worst each timer occupies its own bucket plus a single hash entry. Theoretically this footprint is bounded above by that of the Hashed Hierarchical Timing Wheel, whose multi-level structure can point at a single timer through a chain of wheels. We did not measure a hierarchical wheel directly. Against the single/tickless wheel we benchmarked, Lawn2 uses less memory but the reference Lawn uses more (Section VII).

4) Queueing Theory Sketch: Each TTL bucket, drained by PerTickBookkeeping, is a single-server queue in the formal sense. The standard textbook characterization for such a queue is M/M/1, but its memoryless-service assumption does not fit: a Lawn timer’s expiry is deterministic (a fixed TTL), not exponentially distributed, so the appropriate target is an M/D/1 model, not M/M/1. The Pollaczek-Khinchine result for M/D/1 gives expected queueing delay $E[W_{M/D/1}] = \rho/(2\mu(1 - \rho))$, exactly half the M/M/1 delay $E[W_{M/M/1}] = \rho/(\mu(1 - \rho))$ at the same utilization $\rho = \lambda/\mu$ [14], so an M/M/1 sketch would have overstated, not understated, expected queueing delay here. Even M/D/1 is only a first approximation: PerTickBookkeeping clears an entire due bucket in one batch each tick rather than serving timers one at a time, so the honest target is a bulk-service queue, not a single-item-service one. We leave the bulk-service derivation and its empirical validation to future work (Section VIII-B) rather than state a bound that does not match how a bucket is actually drained. The claims in this paper rest on the measured, not modeled, time and space bounds in Section VII.

D. Memory Access Patterns

Lawn organizes timers by TTL value rather than expiration time, creating a natural grouping of timers with the same TTL. Each bucket is a queue, giving

sequential access when a bucket is drained. The timer index provides $O(1)$ lookups at the cost of one indirection, a common pattern also used by the Hashed Hierarchical Timing Wheel[3]. We did not instrument cache behavior with hardware counters, so we make no measured cache-locality or fragmentation claim. The memory results in Section VII are footprint measurements only. Lawn2 further reduces overhead by embedding nodes in client objects (object pooling by construction) and by avoiding per-entry allocation in its TTL table.

E. Concurrency Considerations

Because TTL buckets are independent, a Lawn can be partitioned across workers, threads, or machines by bucket, and a partitioned deployment needs no synchronization across buckets. We evaluate this honestly in Section VII: on a single multi-core machine, wrapping each partition in its own lock, Lawn2 keeps a modest throughput edge over the wheel but no implementation scales up with thread count under coarse locking. We did not implement a lock-free or per-bucket-lock timer. A genuinely concurrent Lawn is left as future work, and we make no lock-free superiority claim.

V. Comparison and Reflection

General-purpose systems, or applications with highly predictable needs, may prefer the stable run-time of the Timing Wheel (which is asymptotically optimal). Large-scale serving systems, however, suffer from the overflow problem under unpredictable usage. Lawn trades a dependence on the distinct-TTL count for an unbounded span with no rebuilds. It is competitive with, and often faster than, the state of the art only within its assumed regime of low TTL variance (Section VII). It is not a universal improvement.

Operation	TW (Resize)	TW (Multi-Pass)	Lawn
StartTimer	$O(n)^*$	$O(1)$	$O(1)^\dagger$
PerTick	$O(1)$	$O(n)^\ddagger$	$O(t)^\S$
DeleteTimer	$O(1)$	$O(1)$	$O(1)^\dagger$
TimerExpired	$O(1)$	$O(1)$	$O(1)$
space	$O(n)$	$O(n)$	$O(n)$

* on overflow, otherwise amortized $O(1)$. † amortized, worst case $O(n)$ on a resize. ‡ worst case, from the multi-pass cascade. § $O(1)$ empty tick and mean when $t = O(1)$, $O(n)$ worst case.

TABLE IV
Runtime complexity comparison. The Multi-Pass $O(n)$ penalty falls on PerTick (the cascade), consistent with Alg. 2.

VI. Limitations

Lawn’s advantages are conditional, so understanding the limits is essential to choosing it.

A. TTL Distribution Sensitivity

Lawn’s performance depends directly on the distribution of TTL values. When TTLs are drawn from a continuous or highly variable distribution, the number of buckets t grows toward the number of timers n ,

and PerTickBookkeeping approaches $O(n)$. Section VII measures the crossover: Lawn2 loses to the wheel once the distinct-TTL fraction t/N exceeds roughly 0.71–6.4% (tightening monotonically as N grows from 10^4 to 10^7), and is more than two orders of magnitude slower when nearly every timer has a distinct TTL.

B. Memory Overhead for Small Timer Counts

At small scale, Lawn2’s fixed per-structure overhead dominates: at one thousand timers it uses more memory per timer than the wheel, and the advantage appears only at scale (Section VII).

C. Time Granularity and Implementation Complexity

Hierarchical wheels handle variable tick granularity more naturally, level by level, which may suit applications needing very fine-grained precision. Lawn’s mixture of a hash table and per-bucket queues is simple in concept but requires care to implement efficiently (dense layout, avoiding cache misses), especially in a distributed setting.

D. Measurement Ceiling for the Mixed-Operation Test

The realistic mixed-workload test (Section VII-E) initially could not be extended past one million background timers on the 24 GB test machine: past that point, the benchmark process’s own memory footprint left too little headroom, and the operating system began swapping, confirmed directly by inspecting kernel swap-activity counters immediately before and after individual runs. After freeing memory used by other processes on the test machine, we re-ran the same check and confirmed zero swap events up to one hundred million timers, and extended the sweep to that range (Section VII-G). This remains a single-machine measurement, sensitive to whatever else is running on it at the time, not a guarantee that a differently loaded machine, or this one at a different moment, would show the same headroom at the same scale.

VII. Experimental Evaluation

To validate the theoretical analysis and compare the performance of Lawn with Timer Wheel implementations, we conducted an experimental evaluation across various workloads and system configurations.

A. Methodology

We benchmark four C implementations behind a unified interface, all available in the project repository [15]: the reference Lawn (src/lawn.c), the optimized Lawn2 (src/lawn2.c), a production-grade tickless Hierarchical Timing Wheel (Timer Wheel, William Ahern’s `time-out.c` [12]), and a Naive single-level growing ring. All implementations execute on a deterministic, wall-clock-free integer logical clock. A differential correctness gate is run first: all four implementations must produce byte-identical per-tick expiry schedules (with and without deletions, and for large TTLs that force wheel cascades)

across fixed, discrete, and continuous TTL distributions before any measurement is accepted.

Every wheel result in this paper refers specifically to this one cascading, tickless hierarchical implementation, not to timing wheels as a class. To test that scoping claim directly, we implemented and differentially verified two additional baselines: a non-cascading hierarchical wheel modeled on the Linux 4.8+ kernel timer-wheel redesign (direct absolute-position placement at insertion, cascading only when a level’s own granularity boundary is crossed) and a binary min-heap keyed by absolute expiry. At the main baseline (100 000 timers, TTL span 1024, 100 distinct TTLs), the non-cascading wheel inserts in 3.3 ns against the cascading wheel’s 11.4 ns, and its empty-tick cost is 9.3 ns against 141.4 ns, with far lower variance throughout (± 20.6 ns std versus ± 393.0 ns). At ten million timers the empty-tick gap widens to $10.5 \mu\text{s}$ against $132.7 \mu\text{s}$. The empty-tick cost measured elsewhere in this paper is therefore specific to the cascading design, not an inherent property of TTL-indexed wheels in general. The heap is faster still on the empty tick (1.6 ns and 3.4 ns at the same two scales) but far slower at draining due timers ($27.8 \mu\text{s}$ against the non-cascading wheel’s $0.75 \mu\text{s}$ at the 100 000-timer baseline), the expected trade of an $O(\log n)$ structure with no batching. Per-timer memory at one million timers is smallest for the heap (25.2 B), then the non-cascading wheel (32.4 B), then the cascading wheel (88.4 B). None of this changes Lawn2’s own numbers or its crossover boundary, it confirms only that “the wheel” elsewhere in this paper names one specific, cascading implementation, and that a differently engineered wheel occupies a materially different point in the design space.

The evaluation measures scale up to 2×10^8 concurrent timers across three key TTL distribution regimes: fixed ($t = 1$), discrete ($t \ll n$), and continuous uniform ($t \approx n$). Primary evaluated metrics are insertion latency, random deletion cost, per-tick bookkeeping duration, memory footprint, and multi-core scalability.

Each operation is timed with the finest monotonic clock available (CLOCK_UPTIME_RAW, ~ 41 ns quantized on the test machine), so sub-40 ns operations are timed in micro-batches of 256 to lift them off the clock floor. Measurements use fixed seeds (base 1234) with 7 runs and 2 warmup runs (3/1 at scales of 10^6 and above). Each measurement point runs in its own forked child process, so no two points share allocator or cache state. The C results below are authoritative for absolute cost and cross-implementation ranking. We also maintain a Python suite. It is useful for confirming scaling shape, but its absolute numbers carry interpreter overhead and invert insertion rankings, so Python figures are excluded from formal reporting.

Hardware and environment: Apple M4 Pro (12 cores: 8 performance + 4 efficiency), 24 GB RAM, macOS 26.3, Apple clang 21.

B. Basic Operations

We evaluate insertion, deletion, and the two tick paths across scales of one thousand to one million timers.

Naive wins several of the metrics below, which is worth explaining once before the operation-by-operation results, rather than as a series of isolated asides. Naive is a single-level ring sized to the largest TTL it currently holds: while that span stays small, insertion, an empty tick, and expiry all resolve to one direct array index, with no bucket scan and no cascade. Every benchmark in this subsection holds the span small by construction: the operation baselines fix a TTL span of 1024 ticks (chosen by a dedicated sweep described in Section VII-G, not picked arbitrarily), and the expiry test narrows it further to a 200-tick window. Under that condition naive has no bucket-scan or cascade cost to pay at all, so it is not merely competitive, it is the fastest structure measured.

That condition is exactly what Figure 6 varies, and it is exactly where naive fails. Once the TTL span is not known in advance and grows, naive’s ring must grow to cover it, which means rehashing every live timer into the larger ring: an $O(n)$ cost of the same kind as the wheel-resize overflow algorithm (Algorithm 1). The tickless wheel avoids that rehash by paying a bounded cascade cost instead, visible in Figure 11 as the wheel’s own cost growing with scale, and Lawn/Lawn2 avoid it entirely by bucketing on TTL value rather than absolute time. Naive is included as the textbook overflow victim, not as a production candidate: unlike the tickless wheel, a deployed library used as the production baseline throughout this paper, naive exists specifically to make the overflow problem visible. Read the results below accordingly: naive wins wherever the TTL span is small and bounded in advance, and loses precisely when that assumption breaks, which is the condition this paper is about.

1) Insertion: Figure 2 shows insertion latency. Lawn2 is comparable to the wheel at ten thousand timers and about 1.4 to 1.6 times faster at scale (4.9 ns vs. 8.0 ns at one million, and 5.3 ns vs. 7.7 ns at one hundred thousand). The unsorted naive ring is fastest of all (~ 2.3 ns), and the reference Lawn is slower (~ 106 ns at one million, from its per-insert allocation and key copy), which is why the performance claims use Lawn2.

2) Deletion: Figure 3 shows deletion latency. Lawn2 is consistently about 3.2 to 3.8 times faster than the wheel (23.4 ns vs. 88.5 ns at one million, and 4.5 ns vs. 14.3 ns at one hundred thousand), the ratio roughly constant so the absolute gap grows with scale.

3) Expiry cost: Figure 4 shows the cost of a tick that drains due timers, the operation that does the actual work of firing callbacks, using a fixed 200-tick window so every timer is due within it regardless of scale. Lawn2 is about 2.4 times faster than the wheel at one million timers ($303 \mu\text{s}$ vs. $730 \mu\text{s}$): both must inspect every live TTL bucket or wheel level on a draining tick, and Lawn2 does so more cheaply. The naive ring pays no such inspection:

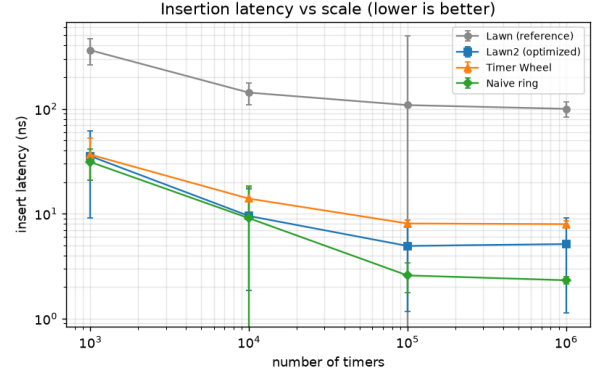


Fig. 2. Insertion latency across scales (C, ns).

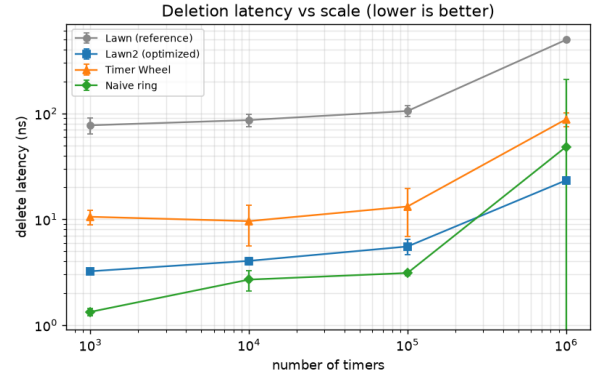


Fig. 3. Deletion latency across scales (C, ns).

it drains its due slot by direct index and so dominates both here ($\sim 5.3 \mu\text{s}$), for the reason given above.

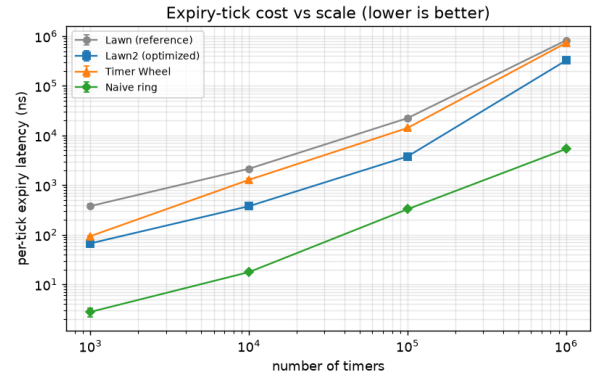


Fig. 4. Per-tick expiry latency across scales (C, ns).

4) Empty-tick (bookkeeping) cost: A tick on which nothing is due does no useful work by itself, but its cost still matters because it runs on every clock tick regardless of load, so a per-tick tax here is paid continuously rather than only when timers actually fire. Lawn2 is flat at $O(1)$ (~ 1 ns) at every scale, while the tickless wheel pays a cascade cost that grows to $\sim 6.6 \mu\text{s}$ at one million timers

(with high variance), more than three orders of magnitude higher. This is the empirical form of the overflow problem: the wheel does work proportional to elapsed time and level structure, Lawn does not. The same gap, and its continuation far past one million timers, is visualized directly in Figures 11 and 14 in Section VII-G: Lawn2 stays at ~ 1 ns independent of t (the *closest expiration guard*) at every scale tested, while the wheel’s cascade cost is unrelated to t and grows with N .

C. Memory Usage

Figure 5 shows per-timer memory at a fixed, modest TTL span of 1024 ticks. At scale Lawn2 uses about 48 bytes per timer versus the wheel’s about 88 (1.8 times less), and grows more slowly. At one thousand timers Lawn2 is heavier (fixed overhead). The reference Lawn uses more than the wheel (~ 113 bytes) at scale. The naive ring is lightest of all here (~ 38 bytes per timer): its cost at this fixed span is just per-timer bookkeeping plus an amortized, doubling array. Figure 6 shows what happens once the span is not fixed: total memory against TTL span, swept from 128 to 524,288 ticks (four-thousand-and-ninety-six-fold) at the main one hundred thousand-timer baseline, and from 128 to 65,536 ticks (five-hundred-and-twelve-fold) at the ten million-timer baseline from Section VII-G. Lawn, Lawn2, and the tickless wheel are flat at both baselines, while the naive single-level ring grows, the overflow rehash described above: 4.2 MB to 29.3 MB (7.0 times) at one hundred thousand timers over the wider span, and 507.5 MB to 510.7 MB at ten million over the narrower range tested there. Section VII-G shows this same growth normalized to each baseline’s own starting point (Figure 13): in relative terms the overflow effect is far weaker at the larger baseline even before this wider span is accounted for.

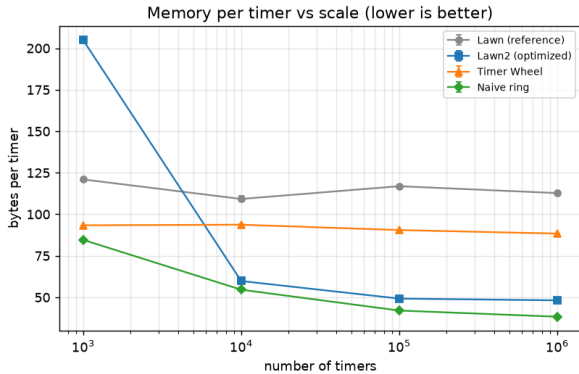


Fig. 5. Memory per timer across scales (C, bytes).

D. Workload Characterization

We measure per-tick expiry cost at one hundred thousand timers under different TTL distributions (Lawn2 vs. the wheel):

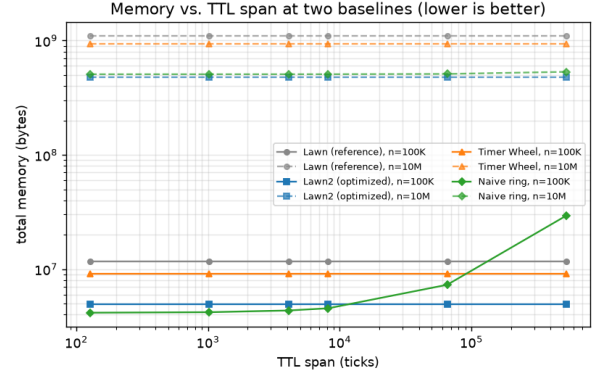


Fig. 6. Total memory vs. TTL span at two baselines, one hundred thousand and ten million timers (C, bytes, lower is better). The naive ring overflows at both while Lawn, Lawn2, and the wheel stay flat.

- Single TTL (all timers share one TTL): 556 ns vs. 11,479 ns ($\sim 20.7\times$).
- Few TTLs (ten distinct values): 3,407 ns vs. 13,325 ns ($\sim 3.9\times$).
- Bursty arrivals: 1,530 ns vs. 10,292 ns ($\sim 6.7\times$).
- Uniform: 3,725 ns vs. 14,056 ns ($\sim 3.8\times$).

Lawn2 is faster on expiry across all of these. The advantage is largest when TTLs cluster, as expected. Figure 7 shows the same comparison directly.

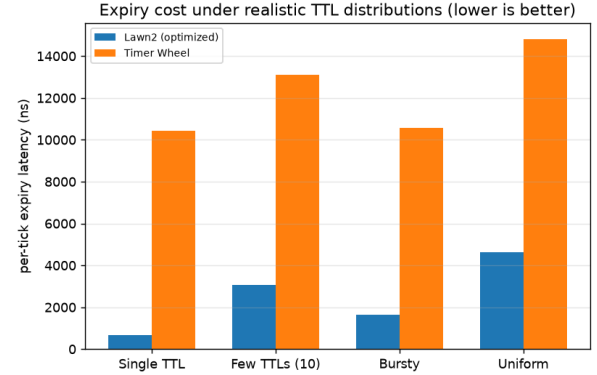


Fig. 7. Per-tick expiry latency, Lawn2 vs. the wheel, under four realistic TTL distributions (C, ns, lower is better).

E. Realistic Mixed-Operation Latency

Every benchmark above isolates one operation type per run: a clean insert-only sweep, a clean tick-only sweep, and so on. A production timer facility never runs that way. Insertions, cancellations, and tick draining interleave continuously. To test whether that isolation hides anything, we ran a realistic mixed workload: pre-fill n background timers, then execute a fixed sequence of five thousand iterations, each draining a tick and, independently, inserting a new timer with 50% probability and cancelling a live timer with 1% probability, timing every individual operation. Figure 8 plots both the mean

and the 99th-percentile latency across this sequence, from one thousand to one million background timers, the range over which we could confirm the measurement itself was not confounded by the test machine’s own memory pressure (Section VI).

At the main baseline (one hundred thousand background timers), Lawn2 beats the wheel clearly on mean latency (97.4 ns vs. 205.5 ns), while the two land close on the 99th percentile (292 ns vs. 208 ns): the wheel’s mean is pulled far above its own p99 by rare, severe stalls rather than by a uniformly higher typical cost, visible in its far larger maximum (186,583 ns vs. 6,083 ns). The reference Lawn and the heap are the next most expensive under this mixed load, both paying for individually-timed operations with no batching, one through per-insertion allocation, the other through $O(\log n)$ restructuring on every insert, delete, and tick. The non-cascading wheel and the naive ring stay cheapest throughout, as expected for the two simplest, most bounded structures tested. The pattern sharpens at one million background timers: Lawn2’s mean and tail both stay essentially flat (102.3 ns mean, 375 ns p99), while the wheel’s mean grows more than twenty-fold (2,152 ns) even though its own p99 barely moves (333 ns), and its maximum reaches 2.1 ms. This is the same scale-driven cascading cost documented independently in Section VII-G, now showing up under a realistic mixed workload rather than an isolated empty-tick sweep. This does not change any of the paper’s central claims, it confirms they survive outside the isolated, single-operation-type setting the rest of this section otherwise uses.

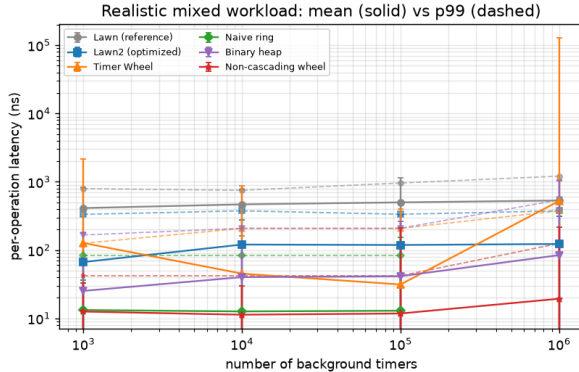


Fig. 8. Per-operation latency under a realistic mixed insert/cancel/tick workload, mean (solid) and p99 (dashed) (C, ns, lower is better).

F. The Distinct-TTL / Timer-Count Crossover

The central caveat of the paper is measured directly in Figure 9, which plots Lawn2’s full-lifecycle speedup over the wheel (Wheel cost divided by Lawn2 cost) as the distinct-TTL fraction t/N varies. Lawn2 wins (speedup above one) while t/N is small, and crosses over to losing

at $t/N \approx 6.4\%$ ($N = 10^4$, about 643 distinct TTLs), 2.2% ($N = 10^5$, about 2 234 distinct TTLs), 1.29% ($N = 10^6$, about 12 920 distinct TTLs), and 0.71% ($N = 10^7$, about 71 407 distinct TTLs). This is the honest boundary of applicability: Lawn is a win precisely when distinct TTLs stay well under a few percent of live timers, comfortably within the range of a configuration-driven or otherwise bounded TTL set. This crossover is about full-lifecycle cost (insert, delete, and tick amortized together), not about any single operation in isolation: even at $t/N = 1$, the extreme case of every timer having a distinct TTL, Lawn2 still drains due timers faster than the wheel (Section VII-D measures this directly). It is specifically the idle empty-tick cost and its amortization over the full lifecycle that flips, not the cost of doing real work. When nearly every timer has a distinct TTL ($t/N = 1$, measured at $N = 10^4$), Lawn2 is up to about 116 times slower than the wheel. The crossover point tightens monotonically across all four measured scales, from 6.4% at $N = 10^4$ down to 0.71% at $N = 10^7$, consistent with the wheel’s own scale-driven cost increase documented independently in Section VII-G: as absolute timer count grows, the wheel’s baseline per-tick cost rises even at a fixed distinct-TTL fraction, so a smaller distinct-TTL fraction is enough to tip the balance. Section VII-G finds a second, independent failure mode of the wheel at far larger N , driven by absolute timer count rather than distinct-TTL count, which we defer there to keep this boundary specific to the axis it measures.

A simple cost model makes the direction of this trend legible, though not its exact shape. PerTickBookkeeping inspects the head of every non-empty TTL bucket, so Lawn2’s per-tick cost is $O(t + k)$ for t active buckets and k expiring timers. A hierarchical wheel’s per-tick cost is $O(L + k)$, where L is its number of levels, a small constant independent of N . Equating the two at parity, $\alpha t \approx \gamma L$ for per-bucket and per-level inspection constants α and γ , predicts a crossover bucket count $t^* \approx (\gamma/\alpha)L$ that does not depend on N , so the crossover fraction t^*/N would simply shrink as $1/N$. The measured crossover counts do not hold constant: 643, 2 234, 12 920, and 71 407 at $N = 10^4$ through 10^7 , growing with N , just more slowly than N itself. A constant- L model cannot produce that growth. This is consistent with, not separate from, Section VII-G’s direct finding that the wheel’s own per-tick cost keeps rising with absolute timer count rather than staying pinned to a small, scale-independent constant.

G. Robustness at Extended Scale

The results above cover one thousand to one million timers, matching the paper’s main methodology. To check that the core claims are not artifacts of that range, we separately swept timer counts far beyond it, at a fixed distinct-TTL count of ten thousand (denser than the main baseline of one hundred), stopping each implementation at the largest count that fit in memory on the 24 GB test machine: Lawn2 and the naive ring to 200 million, and

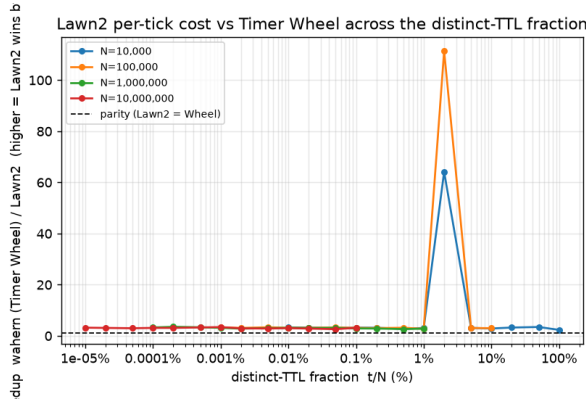


Fig. 9. Lifecycle speedup of Lawn2 over the Timer Wheel (William Ahern’s timeout.c, the cascading implementation defined in Section VII, not the non-cascading wheel introduced later in that same section) vs. the distinct-TTL fraction t/N (C). Higher means Lawn2 wins by more.

the tickless wheel and reference Lawn to 120 million. This is a single-machine check, not a repeated statistical study, so we report it separately from the main results and state only what is unambiguous in the data.

The TTL-span baseline used throughout (1024 ticks) was chosen by a dedicated sweep rather than picked arbitrarily: we measured Lawn2’s and the wheel’s empty-tick cost at one hundred thousand timers across a power-of-two span grid from 128 to 65 536 ticks. The wheel’s cost is genuinely non-monotonic in span, with a reproducible dip near 2048 ticks and a spike near 4096, confirmed by rerunning the sweep independently rather than dismissed as noise. This is consistent with the wheel’s own hierarchical level structure, not a measurement artifact. We chose 1024 ticks as the smallest span past the steep small-span regime that sits on neither the dip nor the spike, and reused the same grid as the span-sweep axis itself (Figures 6, 13, and 14).

Lawn2’s insert, empty-tick, and memory costs stay flat across the entire extended range with no exception. Insert is 4.9 ns at one million timers and 9.2 ns at 200 million. The empty tick is 1.7 ns at one million and 1.3 ns at 200 million. Memory is 48.2 B/timer at one million, converging to 48.0 B/timer by ten million and holding there, with zero measured variance, through 200 million. Figure 10 shows this holds for every implementation tested, not just Lawn2: per-timer memory is flat out to hundreds of millions of timers across the board, in contrast to the scale-driven empty-tick cost below, which is specific to the wheel. The tickless wheel’s empty-tick cost, in contrast, keeps growing with absolute timer count well past the one-million-timer range covered above: 6.6 μ s at one million, 46.1 μ s at ten million, 621.1 μ s at one hundred million, and 747.2 μ s at 120 million. The crossover documented in Figure 9 is therefore not a small-scale artifact: the wheel’s cost continues to grow with absolute timer count at every scale we could test.

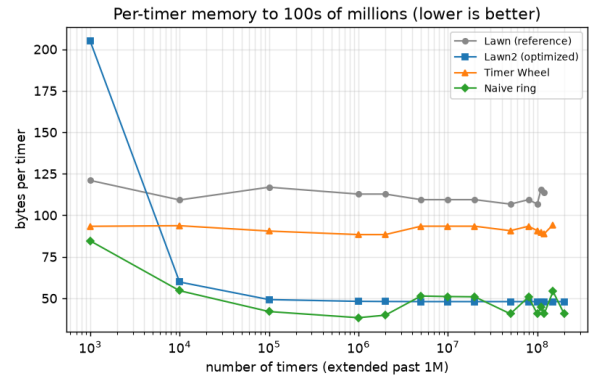


Fig. 10. Per-timer memory vs. timer count, extended to hundreds of millions (C, bytes, lower is better). Flat for every implementation.

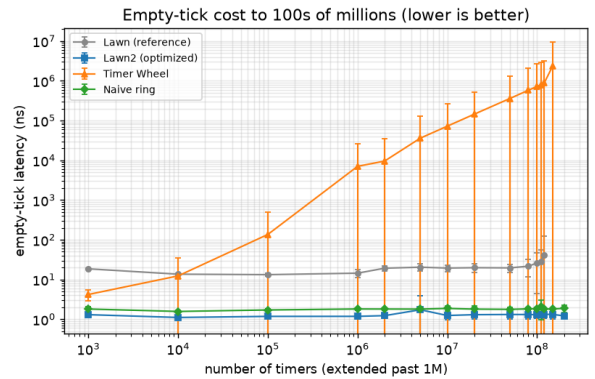


Fig. 11. Empty-tick latency vs. timer count, extended to hundreds of millions (C, ns). Lawn2 stays flat while the wheel’s cost keeps climbing.

The realistic mixed-operation test from Section VII-E shows the same pattern independently. Extended to the largest scale each implementation reaches without exhausting the test machine’s memory (120 million for the wheel and the reference Lawn, 150 million for Lawn2, 200 million for the naive ring, the heap, and the non-cascading wheel), every implementation except the cascading wheel stays within a narrow, roughly flat band (Figure 12): Lawn2’s mean holds between 68 ns and 84 ns across its entire two-million-to-one-hundred-fifty-million range. The wheel’s mean, by contrast, grows almost proportionally with N : 28.1 μ s at ten million, 56.5 μ s at twenty million, 142.7 μ s at fifty million, 298.5 μ s at one hundred million, and 362.7 μ s at 120 million, roughly a thirteen-fold increase for a twelve-fold increase in N . This is the same cascading cost as above, now confirmed under a completely different measurement, a mixed insert/delete/tick sequence rather than an isolated empty-tick sweep, so the two findings independently agree rather than one simply restating the other.

The naive ring’s expiry advantage over Lawn2, conversely, narrows as scale grows and then holds steady

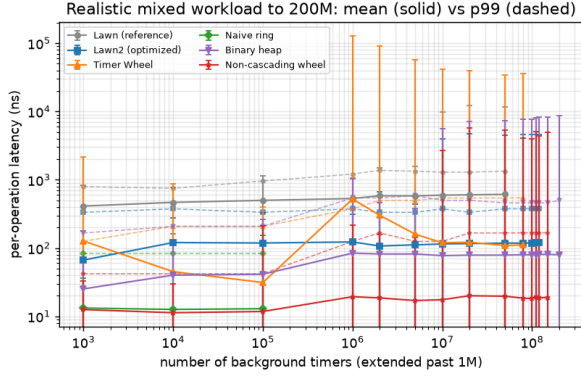


Fig. 12. Realistic mixed-workload latency vs. timer count, extended to 150-200 million depending on implementation (C, ns, mean solid / p99 dashed, lower is better). The wheel’s cost grows with N , every other implementation stays flat.

rather than closing further: 56.9 times faster at one million timers, 29.4 times at ten million, 22.8 times at 50 million, and 21.9 times at one hundred million. One caveat applies across every implementation at this extended scale: deletion latency does not follow a clean trend for any of them, varying by roughly 2 to 12 times between adjacent measured scales with no consistent direction. We attribute this to the interaction between a fully randomized deletion order and cache and allocator state on a very large working set, not to a change in the underlying algorithm, since the same non-monotonic pattern appears in all four implementations rather than only the ones with a case for concern.

Beyond varying timer count alone, we reran the complete parameter sweep, distinct-TTL count, TTL span, and workload, at a fixed baseline of ten million timers instead of the main results’ one hundred thousand, to check whether the crossover and overflow effects documented above scale in the simple way one might expect. They do not, and they diverge in opposite directions. The overflow effect on the naive ring’s memory weakens sharply in relative terms at this baseline. At the original baseline its memory grows 7.0 times across the wider TTL-span sweep (4.16 to 29.3 MB, 128 to 524,288 ticks). At ten million timers the same sweep over its narrower tested range (128 to 65,536 ticks) produces only 0.6% growth, 507.5 to 510.7 MB, because the per-timer cost that dominates total memory at this scale swamps the fixed ring-overhead cost that dominated it at the smaller scale. Figure 13 shows both curves normalized to their own smallest-span point, making the flattening directly visible. This sweep measures steady-state footprint, not the cost of an actual resize event, which moves proportionally more data at larger N and so plausibly grows worse, not better, even as the steady-state tax shrinks. We did not measure a live resize event directly and flag this as a gap rather than a resolved question.

Lawn2’s expiry advantage over the wheel, the real operation of draining due timers rather than the synthetic empty tick, also narrows substantially at this baseline, independently confirmed on two different axes of the same sweep. Across the distinct-TTL sweep, the wheel/Lawn2 ratio at one hundred thousand timers is roughly 3.2 to 3.9 times past a single TTL. At ten million timers the identical sweep falls to 1.8 to 4.0 times, tightening further as distinct-TTL count grows (1.8x at 1000 and 10000 distinct TTLs, versus 3.2 to 3.6x at the same points in the smaller baseline). The workload sweep shows the same narrowing independently: uniform arrivals fall from 4.9x to 2.0x and bursty arrivals from 7.7x to 2.9x between baselines, while spread arrivals hold roughly steady (3.9x to 3.8x). Two independent sweep axes agreeing is what makes this a real effect rather than noise: Lawn2’s margin on real draining work compresses to roughly 2 to 4 times at large N , not the 3.8 to 20.7 times reported for the one hundred thousand-timer baseline in Section VII.

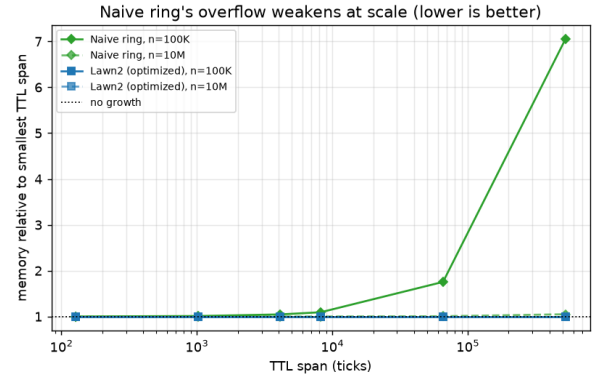


Fig. 13. Naive ring’s memory growth across the TTL-span sweep at two baselines, normalized to each series’ own smallest-span point (C, lower is better). The 1.8x growth at one hundred thousand timers falls to 0.6% at ten million. Lawn2 is flat at both baselines.

The distinct-TTL crossover is, conversely, partly superseded by a second and more severe effect at this scale. At the original baseline the wheel’s empty-tick cost across the distinct-TTL sweep is a mild 3.4 to 153.5 ns, sensitive mainly to how many distinct TTLs are in use. At ten million timers, under the identical sweep, the wheel’s cost balloons to 25.3 to 58.9 μ s and no longer tracks distinct-TTL count in any clean, monotonic way. Lawn2 is unmoved by either axis, holding at about 1.2 to 1.3 ns regardless of distinct-TTL count or absolute scale: its advantage does not depend on which of the two effects, TTL diversity or the wheel’s own scale-driven cost, happens to dominate.

The obvious candidate explanation for the wheel’s cost growing with N at fixed TTL span is density, the number of live timers sharing each tick, N divided by the TTL span. We tested this directly by sweeping the TTL span itself at both baselines and converting to density (Figure 14), rather than inferring it from the distinct-

TTL sweep above. Density alone does not explain the effect. Across the identical span grid (128 to 65 536 ticks), the wheel’s cost at one hundred thousand timers ranges 81.9 to 445.3ns, while at ten million timers it ranges 35.0 to 131.5 μ s, two to three orders of magnitude higher throughout regardless of span. Absolute timer count, not density, is the dominant driver of this cost. Lawn2 is unaffected by either baseline or density, holding at about 1.3ns throughout, the same flat behavior seen against the distinct-TTL sweep above. We did not isolate the specific mechanism behind the wheel’s cost, larger absolute cascade populations, cache locality at a larger working set, or some combination, and flag this as an open question rather than speculate beyond the measurement.

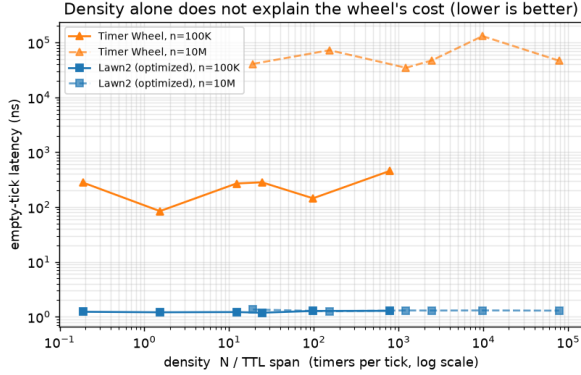


Fig. 14. Empty-tick latency vs. density (timers per tick, $N / \text{TTL span}$) at two baselines (C, lower is better). Matched density does not equalize the wheel’s cost between baselines, so density alone does not explain its blowup. Lawn2 is flat throughout.

H. Concurrency

Figure 15 reports single-machine throughput under an external per-shard mutex applied identically to every implementation. Single-threaded, Lawn2 reaches 121M ops/s versus the wheel’s 90M (the naive ring leads at 134M). As threads are added under coarse locking, aggregate throughput falls for every implementation on this machine (turbo and core-asymmetry effects). Lawn2 retains only a modest ~ 1.3 to $1.4\times$ edge at twelve threads (18.8M vs. 14.1M ops/s). We found no evidence of the large concurrency advantages sometimes attributed to bucketed timers. Realizing one would require a lock-free implementation, which we leave to future work.

VIII. Conclusion

Lawn is a simple, overflow-free timer data structure that maps queues by TTL rather than by expiration time, giving an unbounded span with no rebuilds. Its optimized implementation, Lawn2, beats a production tickless hierarchical timing wheel on deletion ($\sim 3.8\times$ at scale), on insertion at scale (~ 1.4 to $1.6\times$), on per-timer memory at scale ($\sim 1.8\times$), and on draining due timers under every realistic TTL distribution tested (~ 3.8 to

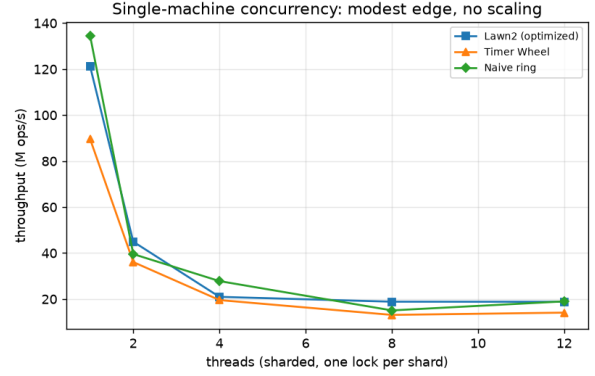


Fig. 15. Single-machine throughput vs. thread count, sharded (C).

$20.7\times$), while keeping memory flat over an unbounded TTL span. The same $O(1)$ bucketing also keeps the idle per-tick check constant, against the wheel’s cascade-bound cost that grows with scale. These gains are real but conditional: they hold only while the number of distinct TTLs stays under roughly two to six percent of the live-timer count, and Lawn degrades to $O(n)$ per tick and loses to the wheel beyond that. Lawn is therefore the right choice for large-scale systems whose timers are drawn from a bounded set of discrete TTLs, and the wrong one for continuous or near-unique TTL distributions.

A. Contributions

- Data structure: the TTL-keyed Queue-Map (Lawn) that removes the overflow problem, plus an allocation-free implementation (Lawn2).
- Analysis: correctness, and complexity with an explicit, measured account of the $O(t)$ per-tick dependence and the empty-tick $O(1)$ fast path.
- Honest evaluation: a differential-gated C benchmark of four implementations across scale, TTL span, distinct-TTL count, and workload, quantifying both where Lawn wins and the t/N crossover where it loses.
- Deployment: Lawn is in production use by several organizations, some independent of the author, including Redis Internals for streams and other internal timers[16], the ReDe Redis dehydration module[17], Mellanox RDMA timers for Unified Communication X[18], and an independent *clib* timer-management utility library, with source available for the ReDe and Unified Communication X implementations.

B. Future Work

- Lock-free / per-bucket-lock Lawn: a genuinely concurrent implementation, and an evaluation on NUMA hardware (not attempted here).
- Formal verification of correctness using theorem provers or model checkers.
- Deterministic-service queueing model (M/D/1) with empirical validation of the latency bounds.

- Distributed implementations for geographically distributed systems.

References

- [1] A. Costello, A. M. Costello, G. Varghese, and G. Varghese, "Redesigning the bsd callout and timer facilities," tech. rep., Washington University in St Louis, 1995.
- [2] A. Lev-Libfeld, "Lawn: an unbound low latency timer data structure for large scale, high throughput systems." arXiv:1906.10860, 2019.
- [3] G. Varghese and A. Lauck, "Hashed and hierarchical timing wheels: efficient data structures for implementing a timer facility," *IEEE/ACM transactions on networking*, vol. 5, pp. 824–834, December 1997.
- [4] D. Sleator and R. Brown, "Calendar queues: A fast $o(1)$ priority queue implementation for the simulation event set problem," *Communications of the ACM*, vol. 31, Oct. 1988.
- [5] D. W. Jones, "An empirical comparison of priority-queue and event-set implementations," *Commun. ACM*, vol. 29, pp. 300–311, Apr. 1986.
- [6] G. Varghese and T. Lauck, "Hashed and hierarchical timing wheels: Data structures for the efficient implementation of a timer facility," *SIGOPS Oper. Syst. Rev.*, vol. 21, pp. 25–38, Nov. 1987.
- [7] S. Scovil and B. C. Nanjundappa, "Time-bucketed balance records: Bounded-storage ephemeral tokens for resource-constrained systems." arXiv:2512.20962, 2025.
- [8] H. Hu, "Large-scale timer management." US Patent no. US8307030B1.
- [9] Z. ZhouIvan, "On-demand scalable timer wheel." US Patent no. US20140298073A1.
- [10] C. Giannoula, F. Strati, D. Siakavaras, G. Goumas, and N. Koziris, "Smartpq: An adaptive concurrent priority queue for numa architectures," 2024.
- [11] A. Drebes, A. Pop, K. Heydemann, N. Drach, and A. Cohen, "Numa-aware scheduling and memory allocation for data-flow task-parallel applications," *SIGPLAN Not.*, vol. 51, Feb. 2016.
- [12] N. Welch, "timeout.c: Tickless hierarchical timing wheel implementation." <http://25thandclement.com/~william/projects/timeout.c.html>.
- [13] A. Lev-Libfeld and A. Margolin, "Fast data - moving beyond from big data's map-reduce," *GeoPython*, vol. 1, pp. 3–6, June 2016.
- [14] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. Wiley-Interscience, 1975.
- [15] A. Lev-Libfeld, "Lawn github repository." <https://github.com/picotera/lawn>.
- [16] S. Iberra, "Redis." <https://redis.io/>, 2009.
- [17] A. Lev-Libfeld, "A redis element dehydration module." <https://github.com/TamarLabs/ReDe>, 2017.
- [18] "Unified communication x." <https://github.com/openucx/ucx/tree/master/src/ucs/time>, 2018.